

Informe Práctica Final Integradora

Helen Valentina Sanabria

Paulina Velásquez Londoño

Yilmar Murillo Jordan

Nicolas Rodríguez

Universidad EAFIT

Análisis y diseño de Algoritmos

Carlos Alberto Alvarez Henao

Mayo 08, 2026

1. Introducción

Un operador de fibra óptica debe resolver simultáneamente tres subproblemas: ordenar y consultar su inventario de solicitudes de servicio, construir la red de menor costo entre sus nodos, y asignar ancho de banda limitado maximizando el ingreso. Cada subproblema tiene una estructura distinta que exige un paradigma algorítmico diferente.

El dataset es el Telco Customer Churn de Kaggle (7.043 registros), donde cada cliente se interpreta como una solicitud de servicio: tenure es la prioridad, MonthlyCharges el valor mensual, TotalCharges el peso en la mochila, y Churn el estado de la solicitud.

El **Módulo A** usa Divide y Vencerás: MergeSort garantiza $O(n \log n)$ en el peor caso con estabilidad, propiedad que QuickSort no ofrece sin modificaciones, y la búsqueda binaria logra $O(\log n)$ sobre el arreglo ordenado. El **Módulo B** usa un algoritmo codicioso porque el MST tiene la propiedad de elección codiciosa: seleccionar siempre la arista de menor peso sin formar ciclo garantiza el óptimo global. El **Módulo C** usa Programación Dinámica porque la Mochila 0-1 carece de esa propiedad: la decisión sobre cada solicitud depende de las demás, y un enfoque codicioso falla.

2. Descripción del dataset

El CSV contiene 7.043 registros y 21 atributos. Se extraen cinco campos: customerID, tenure, MonthlyCharges, TotalCharges y Churn. Los 11 registros con TotalCharges vacío (clientes con tenure = 0 sin cargos acumulados) reciben el valor 0.0.

Propiedad	Valor
Total de registros cargados	7.043
Registros con TotalCharges nulo	11
Registros con Churn= No (activos)	5.174
Registros con Churn = Yes (en riesgo)	1.869
tenure máximo	72
tenure mínimo	0
MonthlyCharges promedio global	64.76 USD

Construcción del grafo (Módulo B): Los registros se reparten en $N = 20$ grupos por índice. El peso de la arista entre los nodos u y v es $c(u,v) = \lfloor \bar{M}_u + \bar{M}_v \rfloor$, donde $\bar{M}_k$ es el promedio de MonthlyCharges del grupo k . El procedimiento es determinista: todos los equipos que usen el mismo dataset obtienen exactamente el mismo grafo de 20 nodos y 190 aristas.

3. Módulo A — Divide y Vencerás

Objetivo: El módulo A procesa un archivo CSV con 7,043 clientes, ordenando los registros por tenure en orden descendente mediante MergeSort y realizando búsquedas binarias recursivas para encontrar valores exactos de tenure.

Procesamiento y Parseo del CSV: El sistema realiza la lectura del archivo: WA_Fn-UseC_-Telco-Customer-Churn.csv. Durante el parseo se procesan correctamente los campos vacíos del atributo TotalCharges, evitando errores de conversión y manteniendo la integridad del dataset.

Métrica	Resultado
Total de registros cargados	7043
Registros con TotalCharges nulo	11
Registros activos (Churn = No)	5174
Registros en riesgo (Churn = Yes)	1869

Implementación de MergeSort

Se utilizó MergeSort debido a su eficiencia para ordenar grandes volúmenes de datos y su comportamiento estable. El algoritmo divide recursivamente el arreglo en dos mitades hasta obtener subarreglos de un elemento y posteriormente los combina en orden descendente según el atributo tenure.

Pseudocódigo de MergeSort

```

MERGESORT(arr, izq, der)
  SI izq >= de
    RETORNAR
  mid ← (izq + der) / 2
  MERGESORT(arr, izq, mid)
  MERGESORT(arr, mid + 1, der)
  MERGE(arr, izq, mid, der)

```

Merge

```

MERGE(arr, izq, mid, der)
  Mientras haya elementos en ambos arreglos:
    SI L[i].tenure >= R[j].tenure
      arr[k] ← L[i]
    SINO
      arr[k] ← R[j]

```

El ordenamiento se realiza de forma descendente.

Pseudocódigo de Búsqueda Binaria

BUSQUEDA_BINARIA(arr, izq, der, k)

SI izq > der

RETORNAR -1

mid $\leftarrow$ (izq + der) / 2

SI arr[mid].tenure == k

RETORNAR mid

SI arr[mid].tenure < k

BUSCAR izquierda

SINO

BUSCAR derecha

Resultado del Ordenamiento

Después de ejecutar MergeSort sobre los 7043 registros:

Resultado	Valor
Mayor tenure	72
Menor tenure	0
Tiempo total de ordenamiento	2.62 ms

Los registros ordenados fueron almacenados en: results/solicitudes_ordenadas.csv

Implementación de Búsqueda Binaria Recursiva: Una vez ordenados los registros, se implementó una búsqueda binaria recursiva para localizar registros cuyo valor de tenure coincida exactamente con un valor k. El arreglo se encuentra ordenado en forma descendente, por lo que las comparaciones se adaptaron a dicho orden.

Análisis de Complejidad: MergeSort: $O(n \log n)$, Búsqueda Binaria: $O(\log n)$

Análisis Empírico de Tiempos

Tamaño de muestra	Tiempo (ms)	$n \cdot \log_2(n)$ normalizado
1000	0.33	1.0000
3500	1.24	4.1347
7043	2.50	9.0333

Resultados de las 5 Consultas

Se realizaron cinco búsquedas binarias utilizando valores exactos de tenure.

Consulta	k	customerID	tenure	Churn
Q_A01	72	6904-JLBGY	72	No
Q_A02	60	7426-WEIJX	60	No
Q_A03	45	7925-PNRGI	45	No
Q_A04	30	2684-EIWEO	30	Yes
Q_A05	12	7450-NWRTR	12	Yes

Los resultados fueron almacenados en: results/busquedas_A.txt

Conclusion: El módulo A permitió implementar exitosamente el paradigma Divide y Vencerás mediante el uso de MergeSort y búsqueda binaria recursiva. Los resultados experimentales confirmaron el comportamiento teórico esperado de: $O(n \log n)$ para el ordenamiento y: $O(\log n)$ para la búsqueda binaria.

4. Módulo B — Codicioso (Kruskal)

A partir del dataset se construyó un grafo completo K_{20} con 20 nodos y 190 aristas. Los promedios de MonthlyCharges por grupo son:

Grupo	Promedio (USD)	Grupo	Promedio (USD)
0	62.88	10	65.57
1	61.30	11	64.48
2	66.18	12	61.66
3	64.83	13	63.85
4	66.07	14	63.64
5	66.56	15	69.05
6	67.71	16	62.10
7	64.69	17	65.67
8	66.42	18	65.50
9	62.89	19	64.19

El costo promedio de arista del grafo completo, calculado como la media de los 190 valores es 129.02 USD.

Pseudocódigo de Kruskal con Union-Find

KRUSKAL(G):

ordenar E por peso ascendente

para cada vértice v en G.V:

MAKE-SET(v)

MST $\leftarrow$ vacío

para cada arista (u, v, w) en E:

si FIND(u) $\neq$ FIND(v):

MST $\leftarrow$ MST $\cup$ {(u, v, w)}

UNION(u, v)

retornar MST

FIND(x):

si padre[x] $\neq$ x:

padre[x] $\leftarrow$ FIND(padre[x]) //compresión de caminos

retornar padre[x]

UNION(x, y):

rx $\leftarrow$ FIND(x)

ry $\leftarrow$ FIND(y)

si rango[rx] < rango[ry]: padre[rx] $\leftarrow$ ry

si rango[rx] > rango[ry]: padre[ry] $\leftarrow$ rx

si rango[rx] = rango[ry]: padre[ry] $\leftarrow$ rx; rango[rx]++

La complejidad total es $O(E \log E)$ debido al ordenamiento de las aristas. Las operaciones Union-Find con compresión de caminos y unión por rango tienen un costo amortizado $O(\alpha(n))$, prácticamente constante.

Resultado del MST — peso total: 2393

El nodo 17 actúa como concentrador central: las 19 aristas del MST conectan todos los demás nodos directamente a él. Agrupadas por peso:

Peso	Nodos conectados al 17	Aristas	Subtotal
124	11, 13	2	248
125	0, 2, 7, 14, 19	5	625
126	1, 3, 6, 9, 10, 15	6	756
127	5, 8, 12, 16, 18	5	635
129	4	1	129

Total**19****2393**

Se puede observar que el nodo 17 actúa como concentrador central del MST, conectándose a todos los demás nodos. Esto ocurre porque el promedio de MonthlyCharges del grupo 17 (65.67 USD) es el que produce los pesos de arista más bajos al combinarse con la mayoría de los otros grupos.

Demostración del Lema del ciclo

Lema: Si e es la única arista de mayor peso en un ciclo C , entonces e no pertenece a ningún MST.

Demostración: Sea $e = (u,v)$ esa arista y T un MST que la contiene. Al eliminar e de T , el árbol se parte en S y $V \setminus S$. Como C es un ciclo, existe otra arista e' en C que también cruza el corte. Como $w(e') < w(e)$, el árbol $T' = T - \{e\} \cup \{e'\}$ tiene menor peso que T , contradicción.

Aplicación concreta: En el ciclo $(11 - 17 - 13)$: aristas $(11,17) = 124$, $(13,17) = 124$ y $(11,13) = \lfloor 64.48 + 63.85 \rfloor = 128$. La arista $(11,13) = 128$ es la de estrictamente mayor peso; Kruskal la descarta porque al evaluarla, 11 y 13 ya están en el mismo componente a través del nodo 17.

Verificación manual — subgrafo de 5 nodos

Matriz de pesos $c(u,v)$:

	0	1	2	3	4
0	—	124	129	127	128
1	124	—	127	126	127
2	129	127	—	131	132
3	127	126	131	—	130
4	128	127	132	130	—

Kruskal selecciona: $(0,1)=124 \rightarrow (1,3)=126 \rightarrow (1,2)=127 \rightarrow (1,4)=127$. Peso total = **504**. El programa produce exactamente estas mismas aristas para los nodos 0 al 4, verificando la correctitud de la implementación.

5. Módulo C — Programación Dinámica

Tabulación (llenado de la tabla dp)

```
MOCHILA_01(n, W, pesos[], valores[]):
    // Inicializar tabla (n+1) × (W+1) en ceros
    dp[0..n][0..W] ← 0

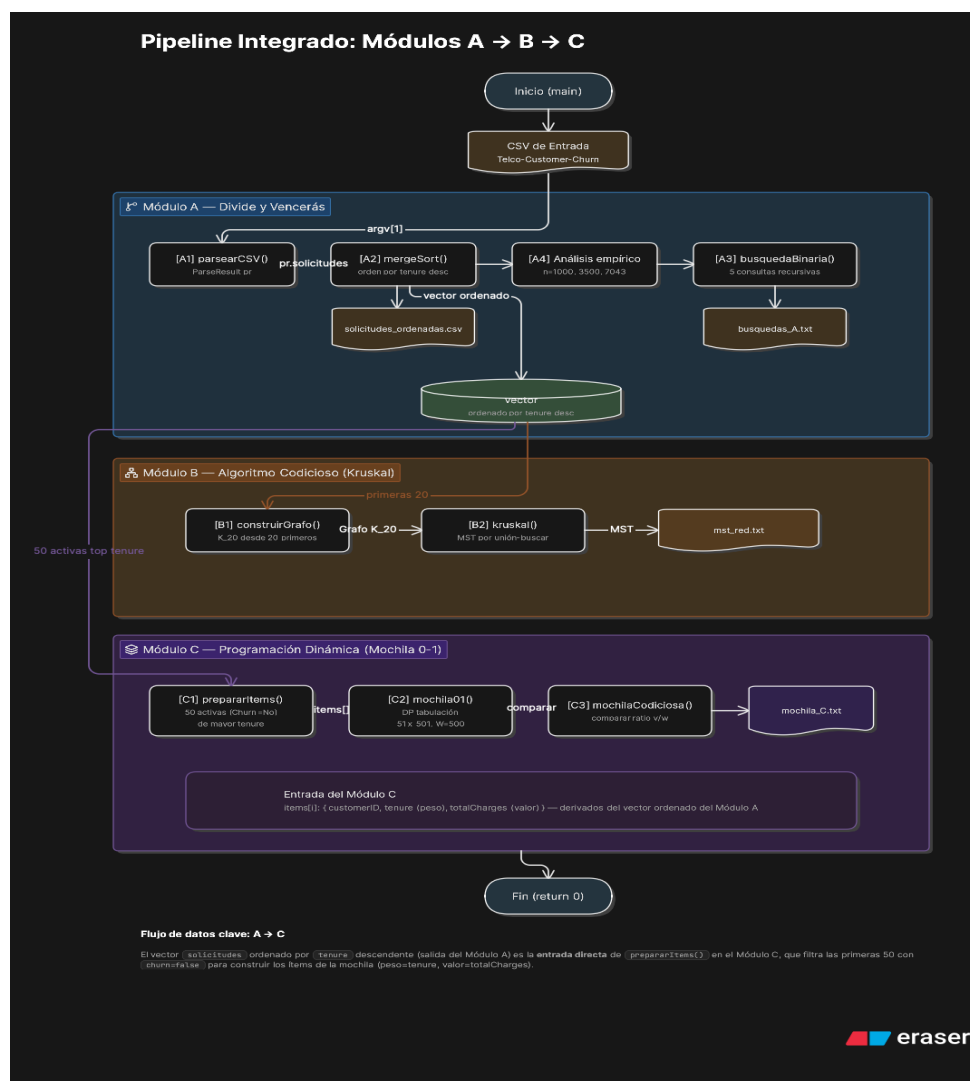
    PARA i ← 1 HASTA n:
        PARA w ← 0 HASTA W:
            dp[i][w] ← dp[i-1][w]           // no tomar ítem i
            SI pesos[i] ≤ w:
                candidato ← dp[i-1][w-pesos[i]] + valores[i]
                SI candidato > dp[i][w]:
                    dp[i][w] ← candidato    // tomar ítem i

    RETORNAR dp[n][W]                       // valor óptimo
```

Backtracking (recuperar ítems seleccionados)

```
BACKTRACK(dp, pesos, n, W):
    w ← W
    seleccionados ← lista vacía
    PARA i ← n HASTA 1 (decrementando):
        SI dp[i][w] ≠ dp[i-1][w]:           // el ítem i fue tomado
            AGREGAR i a seleccionados
            w ← w - pesos[i]
    RETORNAR seleccionados
```


6. Integración del pipeline



7. Conclusiones

Los tres módulos ilustran que no existe un paradigma algorítmico universalmente superior: la elección correcta depende de la estructura del problema.

Módulo	Paradigma	Complejidad	Garantía
A — MergeSort	Divide y Vencerás	$O(n \log n)$	Óptimo y estable
A — Búsqueda binaria	Divide y Vencerás	$O(\log n)$	Exacto sobre arreglo ordenado
B — Kruskal	Codicioso	$O(E \log E)$	Óptimo global (elección codiciosa)
C — Mochila 0-1	Programación Dinámica	$\Theta(n \cdot W)$	Óptimo global (subestructura óptima)

MergeSort confirmó su comportamiento $O(n \log n)$: los tiempos para $n = 1.000$, 3.500 y 7.043 fueron proporcionales a sus funciones $n \log_2(n)$ teóricas. Kruskal reveló que el nodo 17 concentra toda la red, un insight propio de datos reales con distribución estrecha de MonthlyCharges; en instancias sintéticas con pesos aleatorios uniformes esta topología en estrella es improbable. La Programación Dinámica del Módulo C demuestra que cuando las decisiones son interdependientes, el enfoque codicioso produce soluciones subóptimas y la tabulación sistemática es necesaria para garantizar el óptimo, aunque a costa de una complejidad pseudopolinomial en W .

8. Herramientas utilizadas

Se utilizó **Claude (Anthropic, modelo Claude Sonnet)** como apoyo en las siguientes actividades:

- **Módulo A:** Se utilizó IA como apoyo para corregir errores lógicos y verificar la implementación de MergeSort y búsqueda binaria recursiva en C++17 (`mergesort.hpp/cpp`, `binary_search.hpp/cpp`). En particular, se corrigió un error en la búsqueda binaria relacionado con el recorrido del arreglo ordenado de forma
- **Módulo B:** Orientación sobre la estructura de la implementación de Kruskal con Union-Find en C++17 (archivos `graph.hpp/cpp`, `kruskal.hpp/cpp`). El código fue revisado y corregido manualmente; en particular, la construcción del grafo por grupos $i \bmod 20$ fue identificada como incorrecta en una primera versión generada y corregida.
- **Módulo C:** Se utilizó IA como apoyo para el diseño e implementación de la solución de Mochila 0-1 en Python/C++17. En particular, se orientó la estructura de la tabla `dp` de $(n+1) \times (W+1)$ y la lógica de tabulación bottom-up, así como la implementación del backtracking para recuperar los ítems seleccionados. También se apoyó la construcción del contraejemplo que demuestra la falla del enfoque codicioso (3 ítems con $W=10$ donde el algoritmo greedy por ratio v/w obtiene valor 10 frente al óptimo 16 de PD) y la redacción de la discusión sobre pseudopolinomialidad. El mapeo de atributos del dataset a los parámetros de la mochila ($\text{peso} = \lfloor \text{TotalCharges}/100 \rfloor$, $\text{valor} = \text{MonthlyCharges}$) fue definido y verificado por el equipo. Los resultados fueron validados manualmente comparando la solución óptima de PD (755.48 USD, 11 solicitudes) contra la solución codiciosa (753.22 USD, 12 solicitudes).
- **Informe y README:** Apoyo en redacción y estructura.

En todos los casos el uso estuvo acompañado de comprensión y verificación propias. Las decisiones de diseño, corrección de errores y validación de resultados fueron responsabilidad del equipo.

9. Referencias

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press. — Capítulo 23: Minimum Spanning Trees; Capítulo 21: Data Structures for Disjoint Sets.
- [2] Wikipedia contributors. (2025). Kruskal's algorithm. Wikipedia.
https://en.wikipedia.org/wiki/Kruskal%27s_algorithm
- [3] Programiz. Kruskal's Algorithm. <https://www.programiz.com/dsa/kruskal-algorithm>
- [4] IBM Analytics. (2020). Telco Customer Churn dataset. Kaggle.
<https://www.kaggle.com/datasets/blastchar/telco-customer-churn> — Licencia CC0.
- [6] Anthropic. (2026). Claude (modelo Claude Sonnet 4.5). Herramienta de IA generativa utilizada como apoyo en implementación y redacción. <https://claude.ai>

10. Roles del equipo

Paulina Velásquez Londoño: Responsable del diseño, desarrollo y documentación del Módulo A de manera integral.

Helen Valentina Sanabria: Responsable del diseño, desarrollo y documentación del Módulo B en su totalidad.

Yilmar David Murillo y Nicolás Rodríguez: Trabajaron de manera conjunta en el Módulo C. Ambos participaron activamente en las tareas de diseño, desarrollo y documentación del módulo, por lo que la contribución se considera compartida y equitativa.